

Լաբորատոր Աշխատանք 11. Ընդլայնված Օբֆուսկացիա

1. VM-based Obfuscator (Custom Bytecode)

Այստեղ մենք ստեղծում ենք վիրտուալ պրոցեսոր, որն ունի իր սեփական հրամանների համակարգը: Սա թաքցնում է իրական ալգորիթմը վերլուծողից:

Python

```
# Custom Bytecode Definition
```

```
# 0x01: PUSH, 0x02: ADD, 0x03: PRINT, 0x04: HALT
```

```
def virtual_machine(bytecode):
```

```
    stack = []
```

```
    pc = 0 # Program Counter
```

```
    while True:
```

```
        instruction = bytecode[pc]
```

```
        if instruction == 0x01: # PUSH
```

```
            pc += 1
```

```
            stack.append(bytecode[pc])
```

```
        elif instruction == 0x02: # ADD
```

```
            a = stack.pop()
```

```
            b = stack.pop()
```

```
            stack.append(a + b)
```

```
        elif instruction == 0x03: # PRINT
```

```
            print(f"VM Output: {stack.pop()}")
```

```
        elif instruction == 0x04: # HALT
```

```
            break
```

```
        pc += 1
```

```
# Ծրագիրը՝ (5 + 10) հաշվելու համար  
program = [0x01, 5, 0x01, 10, 0x02, 0x03, 0x04]  
virtual_machine(program)
```

2. Control Flow Flattening Generator

Այս մեթոդը քանդում է կողի տրամաբանական կառուցվածքը (loops/if-else) և վերածում այն մեկ հարթ switch մեխանիզմի:

C++

```
#include <iostream>
```

```
void flattened_function(int input) {  
    int state = 0xABC1; // Սկզբնական կամայական վիճակ  
    int x = input;  
  
    while (state != 0) {  
        switch (state) {  
            case 0xABC1:  
                x += 2;  
                state = 0xDEF2;  
                break;  
            case 0xDEF2:  
                if (x % 2 == 0) state = 0x1233;  
                else state = 0x4564;  
                break;  
            case 0x1233:
```

```

std::cout << "Even logic branch\n";

state = 0; // Exit

break;

case 0x4564:

std::cout << "Odd logic branch\n";

state = 0; // Exit

break;

}

}

}

```

3. Real Polymorphic Engine (Python Concept)

Պոլիմորֆիկ Էնջինը ամեն անգամ գեներացնում է նոր կոդ, որն անում է նույն բանը, բայց ունի տարբեր ստորագրություն (signature):

Python

```
import random
```

```
def generate_polymorphic_adder(a, b):
```

```
    # Նույն գործողությունը կատարող տարբեր տարբերակներ
```

```
    variants = [
```

```
        f"print({a} + {b})",
```

```
        f"x = {a}; y = {b}; print(x + y)",
```

```
        f"stack = [{a}, {b}]; print(sum(stack))",
```

```
        f"def tmp(): return {a} + {b}\nprint(tmp())"
```

```
    ]
```

```
    code = random.choice(variants)
```

```
print("--- Generated Polymorphic Code ---")  
print(code)  
exec(code)
```

```
generate_polymorphic_adder(10, 5)
```

4. Anti-VM + Anti-Debugging Combo (C++)

Սա համակցված պաշտպանություն է, որը ստուգում է՝ արդյո՞ք ծրագիրը աշխատում է վիրտուալ մեքենայում (VMware/VirtualBox) կամ Debugger-ի տակ:

C++

```
#include <iostream>
```

```
#ifdef _WIN32
```

```
#include <windows.h>
```

```
#include <intrin.h>
```

```
bool is_debugger_present() {  
    return IsDebuggerPresent();  
}
```

```
bool is_vm() {  
    // Ստուգում ենք CPUID-ի միջոցով "Hypervisor" բիթը  
    int cpuinfo[4];  
    __cpuid(cpuinfo, 1);  
    return (cpuinfo[2] & (1 << 31)); // 31-րդ բիթը ցույց է տալիս վիրտուալացումը  
}
```

```
int main() {  
    if (is_debugger_present() || is_vm()) {  
        std::cout << "Security Violation! Environment not trusted.\n";  
        return -1;  
    }  
  
    std::cout << "Safe environment detected. Running core logic...\n";  
    return 0;  
}  
#endif
```

Ամփոփում (Եզրակացություն)

Այս լաբորատոր աշխատանքի շրջանակներում իրականացվեցին պաշտպանության առաջադեմ մեթոդներ.

1. **VM-based Obfuscation:** Իրական հրամանները վերածվեցին անհասկանալի բայթերի:
2. **Control Flow Flattening:** Ծրագրի ճյուղավորումները թաքցվեցին մեկ ընդհանուր ցիկլի մեջ:
3. **Polymorphic Engine:** Ստեղծվեց դինամիկ կոդի գեներացիայի մեխանիզմ:
4. **Anti-Analysis:** Ներդրվեցին մեխանիզմներ վիրտուալ և debug միջավայրերը հայտնաբերելու համար: